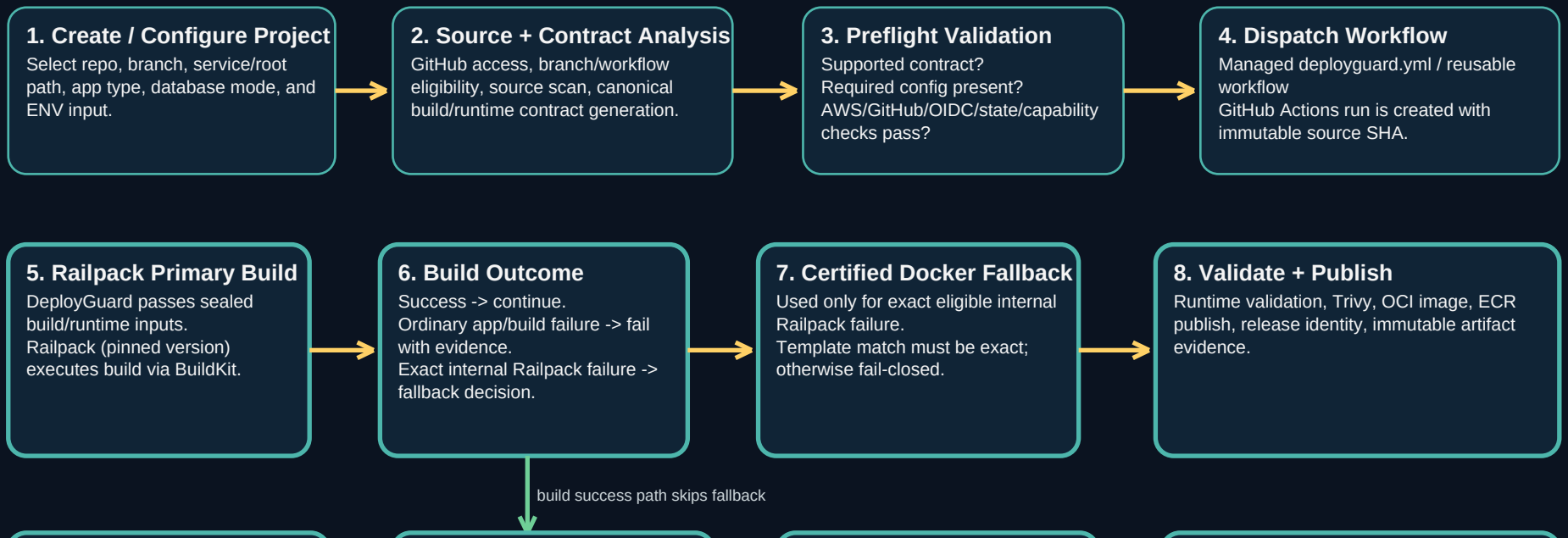
DeployGuard End-to-End Workflow

Complete workflow for project creation, Railpack build, deployment lifecycle, ENV/database handling, failure paths, and operations.



ENV / Database behavior

If app needs no ENV, deployment proceeds normally. If a required user-supplied ENV is missing, preflight or runtime validation fails with a configuration blocker. Managed Database = platform provisions DB and injects connection values. Managed ENV Database = user provides DB ENV values; missing values block or fail verification.

Failure handling

Every failed operation is stored as historical evidence. Current project state is determined by the latest authoritative operation. AI Troubleshooting analyzes failed operations only.

Primary stages

Prepare Source -> Build Application -> Publish Image -> Deploy Runtime -> Verify Application -> Finalize Release. Stages remain sequential and synchronized.

ENV and Database Decision Paths

How DeployGuard behaves when ENV/database configuration is present, absent, or incomplete.

Scenario	What user/platform provides	Expected system behavior
App-only / default database path	No managed database selected. App may have zero required ENV vars.	Project proceeds to build/deploy only if app starts and health checks pass, release becomes LIVE.
Managed Database	User selects Managed Database.	Platform provisions the DB resource, injects connection details/secrets, deploys runtime, then verifies app health.
Managed ENV Database	User selects Managed ENV Database and supplies database ENV values.	DeployGuard validates input presence, passes ENV into runtime, and verifies application startup/health.
Required ENV missing and detectable before dispatch	A required input for selected mode is empty or structurally invalid.	Deployment is blocked early with configuration evidence; GitHub Actions/AWS deployment may not start.
Required ENV missing but only discovered at runtime	Repo starts with hidden runtime dependency on ENV not explicitly provided.	Build may succeed, runtime/startup/health verification fails, evidence is captured, operation is marked FAILED.
Wrong DB credentials / unreachable DB	Values exist but are wrong or target is unavailable.	Build may succeed, runtime verification fails, logs and failure evidence are stored.

Recommended interpretation for teammates

DeployGuard handles two categories of missing ENV: (1) known/configurable missing input, which should block early; (2) repository-owned hidden requirements, which surface during startup or health verification. This distinction explains why some failures happen before dispatch and others during runtime verification.

Why this matters

A successful build does not guarantee a successful deployment. ENV and database correctness are part of runtime readiness. This is why Verify Application remains an explicit lifecycle stage after image publish and runtime deployment.

Edge Cases and Failure Paths

Operational view of where the workflow can stop and how DeployGuard responds.

Stage	Example edge case	System response
Project setup / source	Repo inaccessible, branch missing, GitHub App not installed	Fail before dispatch with repository-access evidence.
Contract generation	Unsupported framework/repo shape, missing startup command	Fail admission/build-target validation with compatibility evidence.
Capability/dispatch preflight	Stale capability contract, OIDC trust mismatch, workflow contract mismatch	Block dispatch; fix backend/config/auth, then retry.
Railpack build	Application build error, bad dependencies, invalid scripts	Fail build with logs/evidence; no fallback.
Railpack internal defect	Pinned Railpack internal eligible panic / exact certified failure	Route to certified Docker fallback if exact template eligibility matches.
Docker fallback	No exact template match, template integrity mismatch, fallback build fail	Fail closed with explicit fallback evidence.
Publish image	Trivy policy block, ECR push failure	Stop release; persist security or publish evidence.
Deploy runtime	Terraform apply failure, AWS quota/IAM/networking issue	Fail deploy runtime; infrastructure evidence retained.
Verify application	Port/bind issue, app crash, health check fails, missing ENV discovered	Mark failed with runtime evidence and troubleshooting candidate.
Current state vs history	Earlier attempts failed, later attempt succeeded	Latest success becomes project current state; failed attempts remain troubleshooting history.
Rollback / destroy	Historical artifact missing, cleanup permission gap	Operation fails with stage-specific evidence; previous LIVE release remains unchanged until successful rollback/destroy.

Railpack position in the architecture

Railpack is the primary builder inside DeployGuard. DeployGuard remains the authority for canonical build/runtime contracts, capability policy, release identity, deployment lifecycle, verification, rollback, and destroy. Railpack handles build-plan execution through BuildKit; fallback is intentionally narrow and used only for exact eligible internal builder failure conditions.

Teammate Operating Summary

Short reference for explaining the platform during team discussion or handoff.

Key explanation points

1. DeployGuard first analyzes the repository and generates a canonical build/runtime contract.
2. It validates GitHub access, workflow readiness, AWS/OIDC/state alignment, and required inputs before dispatch.
3. Railpack is the primary container build engine; BuildKit executes the build.
4. Ordinary application build failures stop at the build stage and become troubleshooting evidence.
5. Exact eligible internal Railpack failure may activate a certified Docker fallback path.
6. After a successful image build, DeployGuard validates runtime assumptions, runs Trivy, publishes to ECR, and deploys via Terraform/ECS.
7. Verify Application checks health, startup behavior, network reachability, and runtime readiness.
8. Missing known ENV should block early; hidden runtime ENV requirements surface at startup/verification.
9. Current project state follows the latest authoritative operation; failed attempts remain selectable troubleshooting history.
10. Redeploy reruns the lifecycle with a new revision/config. Rollback restores a verified historical release. Destroy removes project-scoped resources.

How to explain ENV briefly

If configuration is complete, deployment continues. If a required input is missing, the platform blocks early when it knows the requirement; otherwise the application reaches runtime verification and fails there with evidence.

How to explain Railpack briefly

DeployGuard decides the allowed contract and deployment policy. Railpack performs the build using a pinned version, which helps keep the builder stable and repeatable.